QS-VC Intranet setup

Windows, macOS and Linux | Updated 7 November 2026

1. Connect to your workspace

Use your organisation's approved office network or VPN. Open the HTTPS workspace address supplied by IT in your browser and sign in with your individual account. If the browser warns about the certificate, stop and ask IT to install the organisation's QS-VC root certificate on your device.

The Intranet page shows your approved workspace and the desktop app downloads. If the portal denies access, ask your administrator to check your account and access expiry. A separately hosted workspace needs its own sign-in and access enforcement.

Desktop app (optional)

The browser is the recommended way to join. The desktop app is offered as a plain ZIP archive for each platform; there is no installer. Download the ZIP for your machine from the Intranet page and compare its SHA-256 value with the one published by IT for that exact release. If the values differ, delete the file and contact IT.

Platform	Download	How to start
Windows 10/11 (x64)	QS-VC-Windows-x64.zip	Extract the folder, then run **QS-VC.exe**.
macOS, Apple silicon	QS-VC-macOS-arm64.zip	Extract, move **QS-VC.app** to Applications, open it.
macOS, Intel	QS-VC-macOS-x64.zip	As above. Check the chip in About This Mac.
Linux (x64)	QS-VC-Linux-x64.zip	Extract, then run **./qs-vc** from that folder.

Check the download before opening it:

```
Windows:  Get-FileHash .\QS-VC-Windows-x64.zip -Algorithm SHA256
macOS:    shasum -a 256 ~/Downloads/QS-VC-macOS-arm64.zip
Linux:    sha256sum ~/Downloads/QS-VC-Linux-x64.zip
```

These builds are not code-signed or notarised, so Windows SmartScreen or macOS Gatekeeper may warn on first launch. Only continue when the checksum matches and your IT policy allows it; otherwise use the browser.

Before joining

Check microphone and camera in the meeting preview. If access is blocked, review your browser and system permissions. For screen sharing, choose the intended window or screen when prompted.

Meetings are end-to-end encrypted by default. The lock badge in the meeting shows the state; open it to see the meeting security code and compare it with another participant (both must see the same code). Use a current Chrome, Edge, Safari or Firefox. Room-number rooms joined by Poly SIP/H.323 devices show transport encryption instead, because the bridge must decode audio.

Signed-in users can schedule meetings and webinars. Signed-out webinar guests need host approval. Calendar links prefill individual events; changes to those events are managed in the calendar provider.

Operator setup

Local development and production intranet are separate environments.

2. Run a local development checkout

Install Node.js 22 and Docker Desktop, then set the root environment from `.env.example`. Use `POSTGRES_PORT=5434` for the development Compose database when running Node on the host. Run these commands from the repository root in macOS Terminal, Linux or Windows PowerShell:

```
docker compose up -d postgres redis minio coturn
npm ci
npm run db:migrate
node start-intranet.js
```

Open http://localhost:4002/ready and confirm it reports ready, then open http://localhost:5173. If readiness reports a database error, check the host, port, credentials and migration connection. If sign-in returns 401, check that the account exists and is active. `start-intranet.js` is a development launcher only; it provides no production TLS or hardening.

3. Prepare the production intranet host

Use an x86_64 Linux host with Docker Engine and the Compose plugin, persistent storage and split-horizon DNS. Allow these inbound ports from approved enterprise networks only:

Port	Purpose
TCP 443, TCP 80	Web app, APIs and signaling over TLS; HTTP redirect
UDP + TCP 3478	TURN connectivity
UDP 40000–40100	mediasoup SFU media
UDP 49152–49200	TURN relay media

Keep PostgreSQL, Redis, MinIO and service ports 4000–4004 private to the Docker network. Run one signaling instance; live room state is held in memory, so a restart ends live calls.

Copy `.env.production.example` to `.env.production` and replace every `REPLACE_...` value. Set `DOMAIN` (intranet DNS name), `SFU_ANNOUNCED_IP` (LAN address clients reach), and unique values for `JWT_SECRET` (32+ bytes), `SFU_CONTROL_TOKEN` and `TURN_SECRET`. Never commit or package this file.

Build and load the offline bundle

On an internet-connected Linux release machine with Docker, Syft and Grype, build the six images, SBOMs and checksums (the script fails on any high or critical finding), then copy `release/intranet/` to the intranet host:

```
bash scripts/package-intranet-images.sh .env.production release/intranet
# on the intranet host, in /opt/qsvc
sha256sum -c SHA256SUMS
docker load --input qsvc-intranet-images.tar
```

Initialise the database

Start the infrastructure, create the non-superuser application logins (`qs_auth_app`, `qs_meeting_app`; psql prompts for each password), and run the tenant-isolation check. A new database runs all migrations through 018 (row-level security on every tenant table). For an existing database, back it up first and apply pending migrations with the migration-only administrator.

```
C="docker compose --env-file .env.production -f docker-compose.prod.yml"
$C up -d postgres redis minio coturn
$C cp db/create-application-accounts.sql postgres:/tmp/
$C exec -it postgres sh -lc 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
    -f /tmp/create-application-accounts.sql'
$C cp db/verify-access.sql postgres:/tmp/
$C exec -T postgres sh -lc 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" \
    -d "$POSTGRES_DB" -f /tmp/verify-access.sql'
```

Start the application and distribute the root certificate

```
$C up -d --no-build
$C ps
$C exec -T caddy cat /data/caddy/pki/authorities/local/root.crt > qsvc-intranet-root.crt
```

Distribute only `qsvc-intranet-root.crt` through device management. Never export `root.key`. Clients need the trusted certificate for camera and microphone over HTTPS.

Publish the desktop downloads

The “Desktop downloads” workflow publishes the four ZIPs and their `.sha256` files to the GitHub release `desktop-latest`. Copy them into `installers/` next to `docker-compose.prod.yml` (mounted read-only into the auth service), verify them, and restart auth. A missing ZIP only shows “not available yet” on the Intranet page.

```
mkdir -p installers && cd installers   # copy the ZIPs and .sha256 files here
for f in *.zip; do [ "$(sha256sum "$f" | cut -d' ' -f1)" = \
  "$(tr -d '\r' < "$f.sha256" | cut -d' ' -f1)" ] && echo "OK $f"; done
cd .. && $C up -d auth
```

Meeting encryption

Browser meetings use end-to-end encryption by default: ML-KEM-1024 key exchange and ML-DSA-87 signatures (FIPS 203/204) protect the group key, and AES-256-GCM encrypts each audio and video frame, so the SFU only forwards ciphertext. The key is rotated when anyone leaves. If a browser cannot encrypt, the meeting refuses to send media rather than falling back to clear. Persistent room-number and SIP/H.323 rooms use DTLS-SRTP transport encryption because the Poly bridge must mix decoded audio.

Smoke check

After every install or upgrade, from a computer on the users’ network:

```
npm run smoke:deploy -- https://<domain> --ca qsvc-intranet-root.crt
```

It checks TLS, the gateway, signaling-to-SFU discovery, each TURN server (STUN answer and authenticated relay) and the SFU’s announced UDP media address. Exit code 0 means users can connect and carry audio and video; each FAIL line names the setting to fix.

Acceptance checks

Record results for this workspace and release revision.

4. Verify the actual deployment

From a Windows, a Mac and (if in scope) a Linux device, test trusted HTTPS, sign-in and sign-out, camera and microphone permissions, two-way audio and video with three or more people, screen sharing, and reconnect. Check host approval for a signed-out webinar guest. Test remote access through the approved VPN if it is in scope.

Encryption: confirm the lock badge shows end-to-end encryption on every participant, that two devices show the same meeting security code, and that audio still works after one participant leaves (key rotation). Include at least one Safari or Firefox participant alongside Chrome or Edge.

Downloads: download each published ZIP from the Intranet page, confirm its SHA-256 matches the release value, and launch it on the matching platform.

Infrastructure: `npm run smoke:deploy` passes with exit code `0`, `verify-access.sql` reports PASS, and the database, Redis, MinIO and service ports are not reachable from the client VLAN.

Language support depends on the recognition engine, translation direction and available speech voice. Verify captions and audible translated speech independently. An offline claim needs a two-device test with outbound internet blocked and all required language services available locally.

The Poly bridge is opt-in and needs a configured FreeSWITCH deployment, drachtio and a working H.323 path. AT-01 requires one legacy H.323 Polycom endpoint, one SIP Poly endpoint and one QS-VC browser in the same conference. Verify that each can hear and see both others.

For production, record network reachability, TLS, tenant access checks, control API isolation and deployment-specific security assessment evidence. ISO conformity and live Poly interoperability remain unverified until tested on site.

Release records

Record the source revision, image IDs and SBOMs from the release bundle, the SHA-256 of each desktop ZIP, supported operating systems and browsers, workspace address, smoke-check output, test date, reviewer and results. Store the record through your organisation’s approved release process.